DEPARTAMENTO DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo práctico 1: TAD airMiles.

Especificación del TAD airMiles: sistema de fidelización de una aerolínea.

27 de octubre de 2026 — Algoritmos y Estructuras de Datos

Los4Bugueados

Integrante	LU	Correo electrónico
Campos, Felipe	356/20	felippe.1114@gmail.com
Portaluppi, Alejandro	51/19	alejandro_portaluppi@outlook.com
Chopite, Julio	617/24	juliodavidchopite@gmail.com
Rychter, Ariel	1294/21	arirychter@gmail.com

Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta Baja)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Fax: (++54 +11) 4576-3300

http://www.exactas.uba.ar

1. Decisiones, Aclaraciones y Desarrollo.

Al pensar el problema por primera vez, nos dimos cuenta de que podíamos tratar todo como una transacción. Partimos escribiendo dos observadores: uno donde se cargaban las promociones y otro que era un superobservador que consideraba toda operación como una transacción (Transferencias, Acumulaciones, Canjes). Luego tuvimos que rediseñar los observadores, modularizando entre los diferentes tipos de transacciones, principalmente para las acumulaciones, donde teníamos que conectar también el observador de promociones. Esto nos llevó a terminar modularizando todo en cinco observadores para trabajar de forma prolija.

Estos observadores cubren todos los casos. Con el superobservador anterior no se podían cubrir correctamente, principalmente, los procedimientos acumularMillas y resumenTransacciones. Estos observadores permiten realizar todas las operaciones de forma prolija y sin necesidad de hacer aclaraciones especiales.

Finalmente, los observadores que consideramos minimales para especificar cada proc del TAD quedan de la siguiente manera:

El observador promos contiene las promociones activas: es un diccionario que tiene como clave al nombre de la promoción (nombrePromo) y como valor parametrosPromo, que es un struct que tiene un factor y un tope.

El observador transf es el historial de transferencias: es una secuencia de transacciones, donde las transacciones son un struct que tiene un origen (quien envía la transferencia), un destino (quien la recibe) y la cantidad a transferir.

El observador acu es el historial de acumulaciones: es un diccionario que tiene como clave a los usuarios y como valor una secuencia de acumulaciones. Una acumulación es un struct que contiene la distanciaVolada, una tupla con la promoción con la que se vuela y su factor, y las millasGanadas que se calculan mediante la fórmula dada por el enunciado:

$$millasGanadas = distanciaVolada \cdot factorCategoria \cdot factorPromocion$$

El observador canj es el historial de canjes: es un diccionario cuyas claves son usuarios y cuyo valor es una secuencia de enteros que representa las millas que se canjean en cada operación de canje que realiza el usuario.

El observador ref es un diccionario que toma como clave a un usuario y como valor una secuencia de enteros con el elemento 500, correspondientes a las millas ganadas por haber referido a alguien (donde tanto el referido como el referidor ganan sus 500 millas). No nos importa a quién refirió, ni por quién fue referido, sino únicamente el valor que ganó para poder contabilizarlo en millasTotales y millasDisponibles, y sobre todo para poder excluirlo en resumenTransacciones, que excluye las millas ganadas por referidos.

En el asegura del proc registrarUsuario hicimos un predicado seAgregoUsuario(s,u), donde tomamos la decisión de asegurarnos de que todos los usuarios queden registrados en los observadores, inicializando los diccionarios con la clave del nuevo usuario y su valor como una secuencia vacía.

El historial de transacciones del sistema es global y no discrimina por usuario, por lo tanto, no se modifica al registrar un usuario. Lo mismo sucede con las promociones, ya que no tienen relación con el registro de un usuario. Esta aclaración la hacemos dado que el enunciado pide explícitamente que las transacciones del usuario se inicien vacías, pero nuestras transacciones no están discriminadas por un usuario particular, sino que contemplan a los dos involucrados. Por lo tanto, no dependen de un único usuario y no pueden inicializarse como vacías para cada usuario, dado que las planteamos de manera diferente.

En esta línea, el enunciado dice que en todo momento debe garantizarse que las millas disponibles de un usuario no superen sus millas totales, pero por construcción, millasDisponi-

bles = millasTotales - millasPerdidas, por lo tanto siempre ocurre que $millasDisponibles \leq millasTotales$, entonces no hace falta estar verificando continuamente predicados sobre auxiliares dado que están construidos de forma tal que se respeten las consignas del enunciado. Por otro lado, para cumplir que las operaciones que modifican el sistema deben preservar la integridad de los saldos (ningún usuario puede tener saldo negativo), verificamos que las millasDisponibles sean suficientes para realizar la transferencia o canje. Para que se respete que los identificadores de usuario sean números enteros positivos se requieren positivos para ser válidos como entrada, entre otros casos. De esta manera nos aseguramos respetar estos puntos por construcción.

Decidimos poner la palabra tupla antes de las tuplas dado que el símbolo de tuplas, structs, y secuencias son idénticos y no habría forma de diferenciarlos salvo contexto, especialmente para las tuplas puede resultar confuso por lo que agregamos explícitamente la palabra tupla antes de una tupla.

Hay un error en el enunciado de acumularMillas: si un usuario tiene millas totales mayores o iguales a 50.000, se considera de categoría Oro con su factor asociado, y si un usuario tiene millas totales mayores o iguales a 1.000.000, es de categoría Platino. Ahora bien, si un usuario supera o iguala las 1.000.000 millas totales, el usuario es Oro y Platino al mismo tiempo, lo cual sabemos que no es posible, ya que un usuario no puede estar en dos categorías diferentes al mismo tiempo. Tomamos la decisión de resolver este inconveniente del enunciado imponiendo que el factor Oro para el usuario sea válido cuando tenga menos de 1.000.000 de millas totales.

En transferirMillas encontramos un error de enunciado: dos usuarios pueden transferirse entre ellos infinitamente la misma cantidad una y otra vez, y ambos llegar a la categoría Platino. Por recomendación docente, decidimos seguir la directiva del enunciado, permitir el bug y dejar que el programador decida cómo afrontar este error.

En cuanto al tope de una promo, decidimos hacer lo siguiente: tener un tope igual a -1 significa que no hay tope. Esto lo podemos utilizar, por ejemplo, cuando creamos el sistema con el nombre de la promoción No Promo, o cuando creamos una promoción sin tope. El tope es un número que va bajando a medida que los usuarios vuelan con la promoción y acumulan millas, haciendo: $tope - acumuladoGlobalDeMillasConEsaPromo$ hasta llegar a 0. Mientras baja el tope, la promoción está activa y, cuando llega a 0, se elimina del sistema. Tomamos la decisión de validar en el requiere que las millas a volar no superen el tope de la promoción: si el vuelo supera el tope de la promoción, no puede realizarse con esa promoción.

2. Especificación del TAD airMiles

/∗ Por sugerencia del corrector, para no repetir en cada proc que los observadores no involucrados no sufren modificaciones, dada nuestra construcción de observadores, usamos el predicado losDemasObservadoresNoCambian(s, S_0) de forma flexible para indicar ese comportamiento y resumir los predicados del tipo: $s.observador = S_0.observador$ (excepto el/los que sí cambian) /∗

nombrePromo ES string
factor ES $\mathbb{R}$
parametrosPromo ES $\langle factor : \mathbb{R}, tope : \mathbb{R} \rangle$
trans ES $\langle origen : \mathbb{Z}, destino : \mathbb{Z}, cantidad : \mathbb{R} \rangle$
usuario ES $\mathbb{Z}$
acumulacion ES $\langle distanciaVolada : \mathbb{R}, promo : tupla\langle nombrePromo, factor \rangle, millasGanadas : \mathbb{R} \rangle$
categoria ES $enum\{'Bronce', 'Plata', 'Oro', 'Platino'\}$

TAD airMiles {

obs promos : dict$\langle nombrePromo, parametrosPromo \rangle$
obs transf : seq$\langle trans \rangle$
obs acu : dict$\langle usuario, \text{seq}\langle acumulacion \rangle \rangle$
obs canj : dict$\langle usuario, \text{seq}\langle millas : \mathbb{R} \rangle \rangle$
obs ref : dict$\langle usuario, \text{seq}\langle cantidad : \mathbb{Z} \rangle \rangle$

aux millasTotales$(s : airMiles, u : usuario) : \mathbb{R} =$
$sumaDeAcumulaciones(s, u) + sumaDeTransferenciasRecibidas(s, u) + sumaDeReferidos(s, u)$

aux millasDisponibles$(s : airMiles, u : usuario) : \mathbb{R} =$
$millasTotales(s, u) - millasPerdidas(u, s)$

aux sumaDeAcumulaciones$(s : airMiles, u : usuario) : \mathbb{R} =$
$\sum_{i=0}^{|s.acu[u]|-1} s.acu[u][i].millasGanadas$

aux sumaDeTransferenciasRecibidas$(s : airMiles, u : usuario) : \mathbb{R} =$
$\sum_{i=0}^{|s.transf|-1} \text{IfThenElse}\big(s.transf[i].destino = u, s.transf[i].cantidad, 0\big)$

aux sumaDeTransferenciasEnviadas$(s : airMiles, u : usuario) : \mathbb{R} =$
$\sum_{i=0}^{|s.transf|-1} \text{IfThenElse}\big(s.transf[i].origen = u, s.transf[i].cantidad, 0\big)$

aux sumaDeReferidos$(s : airMiles, u : usuario) : \mathbb{R} = \sum_{i=0}^{|s.ref[u]|-1} s.ref[u][i]$

aux sumaDeCanjes$(s : \text{airMiles}, u : usuario) : \mathbb{R} =$
$\sum_{i=0}^{|s.canj[u]|-1} s.canj[u][i]$

aux millasPerdidas$(u : usuario, s : \text{airMiles}) : \mathbb{R} =$
$sumaDeCanjes(s, u) + sumaDeTransferenciasEnviadas(s, u)$

aux cat$(s : airMiles, u : usuario) : categoria =$
$(res = \text{"}Bronce\text{"} \iff (millasTotales(s, u) < 10{,}000)) \ \wedge$
$(res = \text{"}Plata\text{"} \iff 10{,}000 \leq millasTotales(s, u) < 50{,}000) \ \wedge$
$(res = \text{"}Oro\text{"} \iff 50{,}000 \leq millasTotales(s, u) < 1{,}000{,}000) \ \wedge$
$(res = \text{"}Platino\text{"} \iff millasTotales(s, u) \geq 1{,}000{,}000)$

aux factorCategoria$(s : airMiles, u : usuario) : \mathbb{R} =$
$(res = 1 \iff cat(s, u) = \text{"}Bronce\text{"}) \ \wedge$
$(res = 1{,}5 \iff cat(s, u) = \text{"}Plata\text{"}) \ \wedge$
$(res = 2 \iff cat(s, u) = \text{"}Oro\text{"}) \ \wedge$
$(res = 0 \iff cat(s, u) = \text{"}Platino\text{"})$

proc crearSistema() : airMiles {
requiere { $True$ }
asegura {
$res.acu = \{\} \ \wedge$
$res.transf = <> \ \wedge$
$res.canj = \{\} \ \wedge$
$res.ref = \{\} \ \wedge$
$res.promos = \{\text{"}NoPromo\text{"} : \langle 1, -1 \rangle\}$

```
    }
}

proc registrarUsuario(inout s : airMiles, in u : usuario) {
    requiere { s = S_0 ∧ u > 0 ∧ ¬usuarioEnSistema(s, u) }
    asegura {
        seAgregoUsuario(s, u) ∧ losDemasObservadoresNoCambian(s, S_0)
    }
}

pred usuarioEnSistema(s : airMiles, u : usuario) {
    u ∈ claves(s.acu)
}

pred seAgregoUsuario(s : airMiles, u : usuario) {
    s.acu = setKey(S_0.acu, u, <>) ∧
    s.canj = setKey(S_0.canj, u, <>) ∧
    s.ref = setKey(S_0.ref, u, <>)
}

proc registrarUsuarioConReferidor(inout s : airMiles, in ref : usuario, in u : usuario) {
    requiere { s = S_0 ∧ u > 0 ∧ ¬usuarioEnSistema(s, u) ∧ usuarioEnSistema(s, ref)
    }
    asegura {
        seAgregoUsuario(s, u) ∧_L
        seAgregoReferido(s, S_0, u, ref) ∧
        losOtrosRefNoCambian(s, S_0, u, ref) ∧
        losDemasObservadoresNoCambian(s, S_0)
    }
}

pred seAgregoReferido(s : airMiles, S_0 : airMiles, u : usuario, ref : usuario) {
    s.ref[u] = S_0.ref[u] + + < 500 > ∧
    s.ref[ref] = S_0.ref[ref] + + < 500 >
}

pred losOtrosRefNoCambian(s : airMiles, S_0 : airMiles, u : usuario, ref : usuario) {
    (∀usu : ℤ)(usu ∈ claves(S_0.ref) → usu ∈ claves(s.ref)) ∧_L
    (∀usu : ℤ)((usu ∈ claves(S_0.ref) ∧ usu ≠ u ∧ usu ≠ ref) →_L S_0.ref[usu] =
    s.ref[usu])
}

proc acumularMillas(inout s : airMiles, in u : usuario, in distanciaVolada : ℝ, in p :
nombrePromo, ) {
    requiere {
        s = S_0 ∧ distanciaVolada > 0 ∧
        ((usuarioEnSistema(s, u)) ∧ (existePromo(p, s))) ∧_L
        promoDisponeDeMillasParaVuelo(u, distanciaVolada, p, s, S_0)
    }
    asegura {
```

$s.acu = setKey(S_0.acu, u, S_0.acu[u] + +$
$\langle distanciaVolada, tupla\langle p, S_0.promos[p].factor\rangle, millasGanadas(S_0, distanciaVolada, p, u)\rangle)\wedge$

$\big(promoNoTieneTope(p, s, S_0) \vee$
$casoGastaElTopeDeLaPromo(u, distanciaVolada, p, s, S_0) \vee$
$casoModificaElTopeDeLaPromo(u, distanciaVolada, p, s, S_0)\big) \wedge$
$losDemasObservadoresNoCambian(s, S_0)$
}
}

pred existePromo($p : nombrePromo, s : airMiles$) {
$p \in claves(s.promos)$
}

pred promoDisponeDeMillasParaVuelo($u : usuario, distanciaVolada : \mathbb{R}, p : nombrePromo, s : airMiles, S_0 : airMiles$) {
$promoNoTieneTope(p, s, S_0) \vee$
$millasGanadas(S_0, distanciaVolada, p, u) \leq s.promos[p].tope$
}

aux millasGanadas($S_0 : airMiles, distanciaVolada : \mathbb{R}, p : nombrePromo, u : usuario$)) : $\mathbb{R} = distanciaVolada * S_0.promos[p].factor * factorCategoria(S_0, u)$

pred promoNoTieneTope($p : nombrePromo, s : airMiles, S_0 : airMiles$) {
$S_0.promos[p].tope = -1 \wedge s.promos[p] = S_0.promos[p]$
}

pred casoGastaElTopeDeLaPromo($u : usuario, distanciaVolada : \mathbb{R}, p : nombrePromo, s : airMiles, S_0 : airMiles$) {
$\big((S_0.promos[p].tope - millasGanadas(S_0, distanciaVolada, p, u)) = 0\big) \wedge$
$s.promos = delKey(S_0.promos, p)$
}

pred casoModificaElTopeDeLaPromo($u : usuario, distanciaVolada : \mathbb{R}, p : nombrePromo, s : airMiles, S_0 : airMiles$) {
$\big((S_0.promos[p].tope - millasGanadas(S_0, distanciaVolada, p, u)\big) > 0 \ \wedge$
$s.promos = setKey(S_0.promos, p,$
$\langle S_0.promos[p].factor, S_0.promos[p].tope - millasGanadas(S_0, distanciaVolada, p, u)\rangle$
}

proc canjearMillas(inout s : airMiles, in u : usuario, in $millas : \mathbb{R}$) {
requiere {
$s = S_0 \wedge millas > 0 \wedge usuarioEnSistema(s, u) \wedge_L millas \leq millasDisponibles(s, u)$
}
asegura {
$s.canj = setKey(S_0.canj, u, S_0.canj[u] + + < millas >) \ \wedge$
$losDemasObservadoresNoCambian(s, S_0)$
}

}

proc transferirMillas(in o : usuario, in d : usuario, in $m : \mathbb{R}$, inout s : airMiles) : {
  requiere {
    $(s = S_0) \ \wedge \ m > 0 \ \wedge \ (o \neq d) \ \wedge \ usuarioEnSistema(s, d) \ \wedge$
    $usuarioEnSistema(s, o) \ \wedge_L \ disponeDeMillas(o, m, s)$
  }
  asegura {
    $s.transf = S_0.transf + +\langle origen : o, destino : d, cantidad : m\rangle \wedge$
    $losDemasObservadoresNoCambian(s, S_0)$
  }
}

pred disponeDeMillas($u : usuario, millasATransferir : \mathbb{R}, s : airMiles$) {
  $millasDisponibles(s, u) \geq millasATransferir$
}

proc consultarSaldo(in s : airMiles, in u : usuario) : $\mathbb{R}$ {
  requiere { $usuarioEnSistema(s, u)$ }
  asegura { $res = millasDisponibles(s, u)$ }
}

proc consultarCategoria(in s : airMiles, in u : usuario) : categoria {
  requiere { $usuarioEnSistema(s, u)$ }
  asegura { $res = cat(s, u)$ }
}
proc rankingMillas(in $N : \mathbb{Z}$, in s : airMiles) : seq$\langle usuario\rangle$ {
  requiere {
    $N > 0$
  }
  asegura {
    $estaOrdenadoSegunCantidadMillasDisponibles(res, s) \ \wedge$
    $todosUsuariosEstanEnElHistorial(res, s) \ \wedge$
    $resTieneTamañoMenorOIgualQueN(res, s, N) \ \wedge$
    $todosUsuariosDistintos(res) \ \wedge$
    $todosLosNoSeleccionadosTienenMenosOMismasMillas(res, s)$
  }
}

pred estaOrdenadoSegunCantidadMillasDisponibles(res : seq$\langle usuario\rangle, s$: airMiles)
{
  $(\forall i : \mathbb{Z})(0 \leq i < |res|-1 \ \rightarrow_L millasDisponibles(s, res[i]) \geq millasDisponibles(s, res[i+1]))$
}

pred todosUsuariosEstanEnElHistorial(res : seq$\langle usuario\rangle, s$: airMiles) {
  $(\forall i : \mathbb{Z})(0 \leq i < |res| \ \rightarrow_L \ res[i] \in claves(s.acu))$
}

pred resTieneTamañoMenorOIgualQueN(res : seq$\langle usuario\rangle, s : airMiles, N : \mathbb{Z}$) {

$(N \leq |claves(s.acu)| \ \wedge \ |res| = N) \ \vee$
$(|claves(s.acu)| < N \ \wedge \ |res| = |claves(s.acu)|)$
}

pred todosUsuariosDistintos(res : seq$\langle usuario \rangle$) {
$(\forall i : \mathbb{Z})(0 \leq i < |res| \ \rightarrow_L (\forall j : \mathbb{Z})(0 \leq j < |res| \ \wedge \ i \neq j \ \rightarrow_L \ res[i] \neq res[j]))$
}

pred todosLosNoSeleccionadosTienenMenosOMismasMillas(res : seq$\langle usuario \rangle$, s : airMiles)
{
$(\forall u : usuario)(u \in claves(s.acu) \ \wedge \ u \notin res \wedge |res| > 0 \ \rightarrow_L$
$millasDisponibles(s, u) \leq millasDisponibles(s, res[|res| - 1]))$
}

proc resumenTransacciones(in u : $usuario$, in s : $airMiles$) : $\langle total : \mathbb{R}, canjeadas : \mathbb{R}, enviadas : \mathbb{R}, recibidas : \mathbb{R} \rangle$ {
requiere { $usuarioEnSistema(s, u)$ }
asegura {
$(res.total = millasTotales(s, u)) \wedge$
$(res.canjeadas = sumaDeCanjes(s, u)) \wedge$
$(res.enviadas = sumaDeTransferenciasEnviadas(s, u)) \wedge$
$(res.recibidas = sumaDeTransferenciasRecibidas(s, u))$
}
}
proc iniciarPromocion(inout s : airMiles, in $nombre$: string, in $factor : \mathbb{R}$) {
requiere {
$|nombre| > 0 \wedge factor \geq 1 \wedge s = S_0 \wedge$
$\neg existePromo(nombre, s)$
}
asegura {
$s.promos = setKey(S_0.promos, nombre, \langle factor : factor, tope : -1 \rangle) \wedge$
$losDemasObservadoresNoCambian(s, S_0)$
}
}

proc iniciarPromocionConTope(inout s : airMiles, in $nombre$: string, in $factor : \mathbb{R}$, in $tope : \mathbb{R}$) {
requiere {
$|nombre| > 0 \wedge factor \geq 1 \wedge tope > 0 \wedge s = S_0 \wedge$
$\neg existePromo(nombre, s)$
}
asegura {
$s.promos = setKey(S_0.promos, nombre, \langle factor : factor, tope : tope \rangle) \wedge$
$losDemasObservadoresNoCambian(s, S_0))$
}
}

proc finalizarPromocion(inout s : airMiles, in $nombre$: seq$\langle$char$\rangle$) {
requiere {
$|nombre| > 0 \wedge nombre \neq$ "$NoPromo$" $\wedge existePromo(nombre, s) \wedge s = S_0$

```
    }
    asegura {
        s.promos = delKey(S_0.promos, nombre) ∧
        losDemasObservadoresNoCambian(s, S_0)
    }
}

proc elQueTieneMasMillas(in s : airMiles) : usuario {
    requiere {
        |claves(s.acu)| > 0
    }
    asegura {
        usuarioEnSistema(s, res) ∧_L
        tieneMasMillasQueLosDemas(res, s)
    }
}
pred tieneMasMillasQueLosDemas(res : usuario, s : airMiles) {
    (∀u : usuario)(u ∈ claves(s.acu) →_L millasTotales(s, u) ≤ millasTotales(s, res))
}


proc elMasCanjeador(in s : airMiles) : usuario {
    requiere {
        |claves(s.acu)| > 0
    }
    asegura {
        usuarioEnSistema(s, res) ∧_L tieneMasPerdidasQueLosDemas(res, s)
    }
}

pred tieneMasPerdidasQueLosDemas(res : usuario, s : airMiles) {
    (∀u : usuario)(u ∈ claves(s.acu) →_L (millasPerdidas(u, s) ≤ millasPerdidas(res, s)))
}

proc paresTransferidoresExclusivos(in s : airMiles) : seq⟨tupla⟨usuario, usuario⟩⟩ {
    requiere {
        True
    }
    asegura {
        todosLosParesEnResSonExclusivos(res, s) ∧
        enResEstanTodosLosExclusivos(res, s) ∧
        noHayParesRepetidos(res) ∧
        noSeInviertenEnRes(res)
    }
}

pred sonTransferidores(a : usuario, b : usuario, s : airMiles) {
    (∃i : ℤ)(0 ≤ i < |s.transf| ∧_L ((s.transf[i].origen = a ∧ s.transf[i].destino = b) ∨
    (s.transf[i].origen = b ∧ s.transf[i].destino = a)))
```

```
    }

    pred usuarioAsoloLeTransfiereExclusivamenteAlUsuarioB(a : usuario, b : usuario, s :
    airMiles) {
        (∀i : ℤ)(0 ≤ i < |s.transf| →_L (s.transf[i].origen = a → s.transf[i].destino = b))
    }

    pred sonTransferidoresExclusivos(a : usuario, b : usuario, s : airMiles) {
        sonTransferidores(a, b, s) ∧
        usuarioAsoloLeTransfiereExclusivamenteAlUsuarioB(a, b, s) ∧
        usuarioAsoloLeTransfiereExclusivamenteAlUsuarioB(b, a, s)
    }

    pred todosLosParesEnResSonExclusivos(res : seq⟨tupla⟨usuario, usuario⟩⟩, s : airMiles)
    {
        (∀i : ℤ)(0 ≤ i < |res| →_L sonTransferidoresExclusivos(res[i]_0, res[i]_1, s))
    }

    pred enResEstanTodosLosExclusivos(res : seq⟨tupla⟨usuario, usuario⟩⟩, s : airMiles)
    {
        (∀a, b : usuario)(((usuarioEnSistema(s, a) ∧ usuarioEnSistema(s, b)) ∧_L
        sonTransferidoresExclusivos(a, b, s)) →_L (tupla⟨a, b⟩ ∈ res ∨ tupla⟨b, a⟩ ∈ res))
    }

    pred noHayParesRepetidos(res : seq⟨tupla⟨usuario, usuario⟩⟩) {
        (∀i, j : ℤ)((0 ≤ i < |res| ∧ 0 ≤ j < |res| ∧ i ≠ j) →_L res[i] ≠ res[j])
    }

    pred noSeInviertenEnRes(res : seq⟨tupla⟨usuario, usuario⟩⟩) {
        (∀i : ℤ)(0 ≤ i < |res| →_L tupla⟨res[i]_1, res[i]_0⟩ ∉ res)
    }
}
```